

AI Tools for Bug Hunting & Fixing

The integration of AI into the bug detection and fixing process marks a significant leap in software development. Let's delve deeper into the AI tools that are revolutionizing this domain, offering detailed insights into their functionalities, and providing a guide to using them effectively.

Coverity

Coverity provides static code analysis, which means it examines the code without executing it to find security vulnerabilities, bugs, and quality issues.

How to Use:

- Integrate Coverity with your version control system.
- Set up the analysis parameters and rules based on your project's requirements.
- Run Coverity to scan your codebase. It will parse your code and compare it against a database of known issues.
- Review the issues detected by Coverity in its dashboard, where it provides detailed insights and potential fixes.

Fortify

Fortify offers both static and dynamic application security testing. It can identify security vulnerabilities in the code and also monitor the application in runtime for unusual behaviors.

How to Use:

- Install and configure Fortify to work with your codebase.
- Conduct a static scan by running Fortify against your source code.
- Analyze the static scan results to identify and address security concerns.

- Set up Fortify to monitor your application in a dynamic analysis mode during runtime.
- Use the findings from dynamic analysis to patch runtime vulnerabilities.

DeepCode

DeepCode uses deep learning to review code changes and advise on potential issues by learning from the collective knowledge of the global developer community.

How to Use:

- Connect DeepCode with your GitHub, Bitbucket, or GitLab repository.
- Allow DeepCode to analyze your codebase; it will provide suggestions categorized by criticality.
- Act on the recommendations given by DeepCode to prevent potential bugs before they occur.

CodeGuru Reviewer

Amazon's CodeGuru Reviewer is an automated code review service that uses machine learning to identify critical issues, security vulnerabilities, and hard-to-find bugs during application development.

How to Use:

- Integrate CodeGuru Reviewer with your repository hosted on AWS CodeCommit, GitHub, or Bitbucket.
- Create a pull request to trigger the CodeGuru Reviewer.
- Receive automated suggestions in the pull request comments.
- Follow the recommendations to improve code quality and address potential bugs.
- Automated Error Tracking and Reporting

```

298 public PairOfSameType<HRegion> stepsBeforePONR(final Server server,
299         final RegionServerServices services, boolean testing) throws IOException {
300     // Set ephemeral SPLITTING znode up in zk. Mocked servers sometimes don't
301     // have zookeeper so don't do zk stuff if server or zookeeper is null

```

1. Condition `server != null`, taking true branch

2. `alloc_fn`: A new resource is returned from allocation method `getZooKeeper`. (The virtual call resolves to `org.apache.hadoop.hbase.master.cleaner.TestHFileCleaner.DummyServer.getZooKeeper`.)

3. Condition `server.getZooKeeper() != null`, taking true branch

◆ CID 1090416 (#1 of 2): Resource leak (RESOURCE_LEAK)

4. `leaked_resource`: Failing to save or close resource created by `server.getZooKeeper()` leaks it.

```

302     if (server != null && server.getZooKeeper() != null) {
303         try {
304             createNodeSplitting(server.getZooKeeper(),
305                 parent.getRegionInfo(), server.
306         } catch (KeeperException e) {
307             throw new IOException("Failed cre
308                 this.parent.getRegionNameAsStri
309         }
310     }
311     this.journal.add(JournalEntry.SET_SPL

```

```

@Override
public ZooKeeperWatcher getZooKeeper() {
    try {
        return new ZooKeeperWatcher(getConfiguration(), "dummy server", this);
    } catch (IOException e) {
        e.printStackTrace();
    }
    return null;
}

```